Program 1: Multi-threaded Real-time Chat System

Thread priorities, isAlive(), join(), suspend, resume and stop

Program

```
// Practical Lab: Multi-threaded Real-time Chat System Simulation
// Demonstrating isAlive(), join(), Thread Priorities, Suspension, Resumption, and Stopping

class ChatUser extends Thread {
    private String userName;
    private String[] messages;
    private boolean suspended = false;
    private boolean stopped = false;

    // Constructor
    public ChatUser(String userName, String[] messages) {
        this.userName = userName;
        this.messages = messages;
    }

    // Method to safely suspend the thread
    public synchronized void suspendThread() {
        suspended = true;
        System.out.println("[" + userName + "] is temporarily suspended (paused).");
    }

    // Method to safely resume the thread
    public synchronized void resumeThread() {
        suspended = false;
        notify();
        System.out.println("[" + userName + "] has been resumed.");
    }

    // Method to safely stop the thread
    public synchronized void stopThread() {
        stopped = true;
        suspended = false;
        notify();
        System.out.println("[" + userName + "] has been stopped.");
    }

    @Override
    public void run() {
        for (String msg : messages) {
            // Check if thread was stopped
            if (stopped) {
                break;
            }

            // Check if thread is suspended
            synchronized (this) {
                while (suspended) {
                    try {
                        wait();
                    } catch (InterruptedException e) {
                        System.out.println(userName + " was interrupted.");
                    }
                }
            }

            // Send/receive message
            System.out.println("[" + userName + " | Priority " + getPriority() + "]: " + msg);

            try {
                Thread.sleep(500); // Simulate network transmission delay
            } catch (InterruptedException e) {
                System.out.println(userName + " was interrupted.");
            }
```

```
        }

        System.out.println("--> " + userName + " finished conversation.");
    }
}

public class ChatSystemDemo {
    public static void main(String[] args) {
        System.out.println("===== MULTI-THREADED REAL-TIME CHAT SYSTEM =====\n");

        // 1. Create chat user threads with different messages
        String[] adminMsgs = {"CRITICAL ALERT: System backup starting soon!",
            "High-priority announcement delivered."};

        String[] aliceMsgs = {
            "Hello everyone!", "How are you doing today?", "Did anyone see the project
report?"};

        String[] bobMsgs = {
            "Hi Alice!", "I'm working on the assignment.", "Yes, I saw the report."};

        ChatUser admin = new ChatUser("Admin", adminMsgs);
        ChatUser alice = new ChatUser("Alice", aliceMsgs);
        ChatUser bob = new ChatUser("Bob", bobMsgs);

        // 2. Set Thread Priorities (High-priority messages handled first)
        admin.setPriority(Thread.MAX_PRIORITY); // Priority 10
        alice.setPriority(Thread.NORM_PRIORITY); // Priority 5
        bob.setPriority(Thread.MIN_PRIORITY); // Priority 1

        System.out.println("Thread Priorities Configured:");
        System.out.println("Admin Priority: " + admin.getPriority() + " (MAX_PRIORITY)");
        System.out.println("Alice Priority: " + alice.getPriority() + " (NORM_PRIORITY)");
        System.out.println("Bob Priority : " + bob.getPriority() + " (MIN_PRIORITY)\n");

        // 3. Start Threads
        System.out.println("--- Starting Chat Threads ---");
        admin.start();
        alice.start();
        bob.start();

        // 4. Demonstrate isAlive() to check thread status
        System.out.println("\nChecking Thread Status using isAlive():");
        System.out.println("Is Admin active? " + admin.isAlive());
        System.out.println("Is Alice active? " + alice.isAlive());
        System.out.println("Is Bob active?   " + bob.isAlive() + "\n");

        try {
            // Let them run briefly
            Thread.sleep(400);

            // 5. Demonstrate Thread Suspension
            System.out.println("\n--- Demonstrating Thread Suspension ---");
            bob.suspendThread();

            // Allow Alice and Admin to continue chatting while Bob is suspended
            Thread.sleep(800);

            // 6. Demonstrate Thread Resumption
            System.out.println("\n--- Demonstrating Thread Resumption ---");
            bob.resumeThread();

            // 7. Demonstrate Thread Stopping
            Thread.sleep(300);
            System.out.println("\n--- Demonstrating Thread Stopping ---");
            alice.stopThread();

            // 8. Demonstrate join() to ensure proper synchronization
            System.out.println("\n--- Synchronizing using join() ---");
            System.out.println("Waiting for all active users to finish...");
```

```
            admin.join();
            alice.join();
            bob.join();
            System.out.println("All threads have completed their execution.");

            // 9. Verify with isAlive() after join()
            System.out.println("\nFinal Status after join():");
            System.out.println("Is Admin active? " + admin.isAlive());
            System.out.println("Is Alice active? " + alice.isAlive());
            System.out.println("Is Bob active?   " + bob.isAlive());

        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted: " + e.getMessage());
        }

        System.out.println("\n===== CHAT SESSION CLOSED SUCCESSFULLY =====");
    }
}
```

Output

```
>> java ChatSystemDemo
>>
===== MULTI-THREADED REAL-TIME CHAT SYSTEM =====

Thread Priorities Configured:
Admin Priority: 10 (MAX_PRIORITY)
Alice Priority: 5 (NORM_PRIORITY)
Bob Priority  : 1 (MIN_PRIORITY)

--- Starting Chat Threads ---

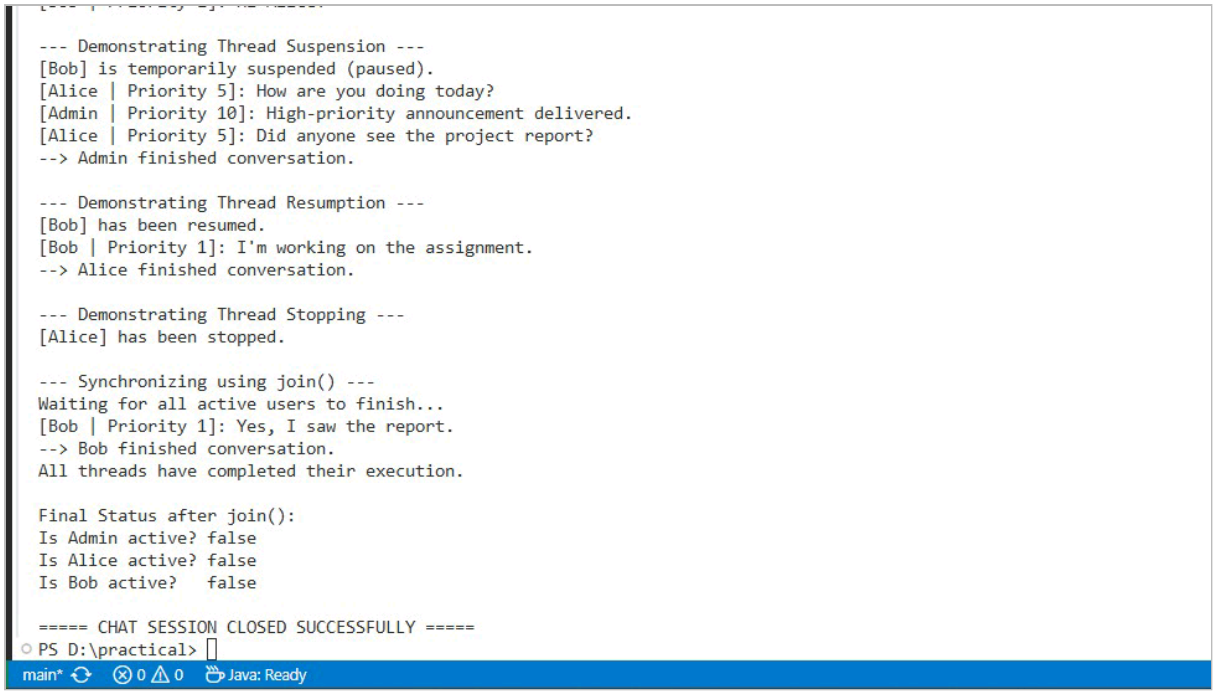
Checking Thread Status using isAlive():
Is Admin active? true
Is Alice active? true
Is Bob active?   true

[Admin | Priority 10]: CRITICAL ALERT: System backup starting soon!
[Alice | Priority 5]: Hello everyone!
[Bob | Priority 1]: Hi Alice!

--- Demonstrating Thread Suspension ---
[Bob] is temporarily suspended (paused).
[Alice | Priority 5]: How are you doing today?
[Admin | Priority 10]: High-priority announcement delivered.
[Alice | Priority 5]: Did anyone see the project report?
--> Admin finished conversation.

--- Demonstrating Thread Resumption ---
[Bob] has been resumed.
[Bob | Priority 1]: I'm working on the assignment.
--> Alice finished conversation.

--- Demonstrating Thread Stopping ---
[Alice] has been stopped.
```


--- Demonstrating Thread Suspension ---
[Bob] is temporarily suspended (paused).
[Alice | Priority 5]: How are you doing today?
[Admin | Priority 10]: High-priority announcement delivered.
[Alice | Priority 5]: Did anyone see the project report?
--> Admin finished conversation.
--- Demonstrating Thread Resumption ---
[Bob] has been resumed.
[Bob | Priority 1]: I'm working on the assignment.
--> Alice finished conversation.
--- Demonstrating Thread Stopping ---
[Alice] has been stopped.
--- Synchronizing using join() ---
Waiting for all active users to finish...
[Bob | Priority 1]: Yes, I saw the report.
--> Bob finished conversation.
All threads have completed their execution.
Final Status after join():
Is Admin active? false
Is Alice active? false
Is Bob active? false
===== CHAT SESSION CLOSED SUCCESSFULLY =====
PS D:\practical>
main* 0 0 Java: Ready

Program 2: ATM Simulation

Exception handling with a menu-driven ATM

Program

```
import java.util.Scanner;

class ATM {
    private double balance;

    // Constructor
    ATM(double initialBalance) {
        balance = initialBalance;
    }

    // Check balance
    void checkBalance() {
        System.out.println("Available Balance = Rs. " + balance);
    }

    // Deposit money
    void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be greater than zero.");
        }

        balance = balance + amount;

        System.out.println("Rs. " + amount + " deposited successfully.");
    }

    // Withdraw money
    void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be greater than zero.");
        }

        if (amount > balance) {
            throw new ArithmeticException("Insufficient funds. Withdrawal cannot be
completed.");
        }

        balance = balance - amount;

        System.out.println("Rs. " + amount + " withdrawn successfully.");
    }
}

public class ATMSimulation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        ATM atm = new ATM(10000);

        int choice;
        boolean running = true;

        System.out.println("===== ATM MACHINE =====");

        while (running) {
            System.out.println("\n----- ATM MENU -----");
            System.out.println("1. Check Balance");
            System.out.println("2. Withdraw Money");
            System.out.println("3. Deposit Money");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");

            try {
                choice = sc.nextInt();
```

```
                switch (choice) {
                    case 1:
                        atm.checkBalance();
                        break;

                    case 2:
                        System.out.print("Enter withdrawal amount: Rs. ");

                        double withdrawAmount = sc.nextDouble();

                        try {
                            atm.withdraw(withdrawAmount);

                        } catch (ArithmeticException e) {
                            System.out.println("Error: " + e.getMessage());

                        } catch (IllegalArgumentException e) {
                            System.out.println("Invalid Input: " + e.getMessage());
                        }

                        break;

                    case 3:
                        System.out.print("Enter deposit amount: Rs. ");

                        double depositAmount = sc.nextDouble();

                        try {
                            atm.deposit(depositAmount);

                        } catch (IllegalArgumentException e) {
                            System.out.println("Invalid Input: " + e.getMessage());
                        }

                        break;

                    case 4:
                        running = false;
                        System.out.println("Thank you for using the ATM.");
                        break;

                    default:
                        throw new IllegalArgumentException("Invalid menu choice.");
                }

            } catch (java.util.InputMismatchException e) {
                System.out.println("Error: Please enter a valid numeric input.");

                sc.nextLine();

            } catch (IllegalArgumentException e) {
                System.out.println("Error: " + e.getMessage());

            } finally {
                System.out.println("Transaction processing completed.");
            }
        }

        sc.close();
    }
}
```

Output

```
PS D:\practical> javac ATMSimulation.java
>> java ATMSimulation
>>
===== ATM MACHINE =====

----- ATM MENU -----
1. Check Balance
2. Withdraw Money
3. Deposit Money
4. Exit
Enter your choice: 1
Available Balance = Rs. 10000.0
Transaction processing completed.

----- ATM MENU -----
1. Check Balance
2. Withdraw Money
3. Deposit Money
4. Exit
Enter your choice: 2
Enter withdrawal amount: Rs. 200
Rs. 200.0 withdrawn successfully.
Transaction processing completed.

----- ATM MENU -----
1. Check Balance
2. Withdraw Money
3. Deposit Money
4. Exit
Enter your choice: 3
Enter deposit amount: Rs. 1000
Rs. 1000.0 deposited successfully.
Transaction processing completed.
```

```
----- ATM MENU -----
1. Check Balance
2. Withdraw Money
3. Deposit Money
4. Exit
Enter your choice: 2
Enter withdrawal amount: Rs. 12345678
Error: Insufficient funds. Withdrawal cannot be completed.
Transaction processing completed.

----- ATM MENU -----
1. Check Balance
2. Withdraw Money
3. Deposit Money
4. Exit
Enter your choice: 3
Enter deposit amount: Rs. -500
Invalid Input: Deposit amount must be greater than zero.
Transaction processing completed.

----- ATM MENU -----
1. Check Balance
2. Withdraw Money
3. Deposit Money
4. Exit
Enter your choice: rffg
Error: Please enter a valid numeric input.
Transaction processing completed.

----- ATM MENU -----
1. Check Balance
2. Withdraw Money
3. Deposit Money
4. Exit
Enter your choice:
```

Program 3: Single Inheritance

Superclass Animal and subclass Dog

Program

```
// Program to demonstrate Single Inheritance

class Animal {
    // Method of superclass
    void eat() {
        System.out.println("Animal is eating.");
    }

    // Method of superclass
    void sleep() {
        System.out.println("Animal is sleeping.");
    }
}

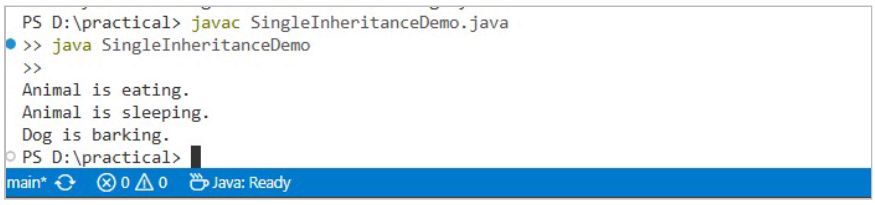
class Dog extends Animal {
    // Method of subclass
    void bark() {
        System.out.println("Dog is barking.");
    }
}

public class SingleInheritanceDemo {
    public static void main(String[] args) {
        // Create object of subclass
        Dog d = new Dog();

        // Calling superclass methods
        d.eat();
        d.sleep();

        // Calling subclass method
        d.bark();
    }
}
```

Output



Program 4: Robust Calculator

Input validation and exception handling

Program

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class RobustCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean continueCalculation = true;

        System.out.println("===== ROBUST JAVA CALCULATOR =====");

        while (continueCalculation) {
            double num1 = 0;
            double num2 = 0;
            char operator;

            // Accept first number
            while (true) {
                try {
                    System.out.print("\nEnter first number: ");
                    num1 = sc.nextDouble();
                    break;
                } catch (InputMismatchException e) {
                    System.out.println("Error: Please enter a valid number.");
                    sc.next();
                }
            }

            // Accept operator
            while (true) {
                System.out.print("Enter operator (+, -, *, /): ");
                String input = sc.next();

                if (input.length() == 1
                    && (input.charAt(0) == '+' || input.charAt(0) == '-' || input.charAt(0) ==
'*'
                        || input.charAt(0) == '/')) {
                    operator = input.charAt(0);
                    break;

                } else {
                    System.out.println("Error: Invalid operator. Please enter +, -, * or /.");
                }
            }

            // Accept second number
            while (true) {
                try {
                    System.out.print("Enter second number: ");
                    num2 = sc.nextDouble();
                    break;
                } catch (InputMismatchException e) {
                    System.out.println("Error: Please enter a valid number.");
                    sc.next();
                }
            }

            double result = 0;
            boolean validOperation = true;

            // Perform calculation
            switch (operator) {
                case '+':
                    result = num1 + num2;
                    break;
```

```
                case '-':
                    result = num1 - num2;
                    break;

                case '*':
                    result = num1 * num2;
                    break;
                case '/':
                    if (num2 == 0) {
                        System.out.println("Error: Division by zero is not allowed.");
                        validOperation = false;
                    } else {
                        result = num1 / num2;
                    }
                    break;

                default:
                    validOperation = false;
                    System.out.println("Error: Invalid operator.");
            }

            // Display result
            if (validOperation) {
                System.out.println("Result = " + result);
            }

            // Ask whether to continue
            while (true) {
                System.out.print("\nDo you want to perform another calculation? (Y/N): ");
                String choice = sc.next();

                if (choice.equalsIgnoreCase("Y")) {
                    break;
                } else if (choice.equalsIgnoreCase("N")) {
                    continueCalculation = false;
                    break;
                } else {
                    System.out.println("Error: Please enter Y or N.");
                }
            }
        }

        System.out.println("\nCalculator terminated successfully.");
        sc.close();
    }
}
```

Output

```
>>
PS D:\practical> java RobustCalculator
>>
===== ROBUST JAVA CALCULATOR =====

Enter first number: 2
Enter operator (+, -, *, /): +
Enter second number: 3
Result = 5.0

Do you want to perform another calculation? (Y/N): n

Calculator terminated successfully.
PS D:\practical> java RobustCalculator
>>
===== ROBUST JAVA CALCULATOR =====

Enter first number: 22
Enter operator (+, -, *, /): /
Enter second number: 0
Error: Division by zero is not allowed.

Do you want to perform another calculation? (Y/N): n

Calculator terminated successfully.
PS D:\practical> java RobustCalculator
>>
===== ROBUST JAVA CALCULATOR =====

Enter first number: aa
Error: Please enter a valid number.

Enter first number: 11
Enter operator (+, -, *, /): *
Enter second number: 5
Result = 55.0

Do you want to perform another calculation? (Y/N): n

Calculator terminated successfully.
main*  ⊗ 0 ⚠ 0  Java: Ready
```

Program 5: Hotel Room Booking System

Floors, rooms and booking status

Program

```
import java.util.Scanner;

public class HotelRoomBooking {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // 3 floors and 5 rooms on each floor
        char[][] rooms = new char[3][5];

        // Initialize all rooms as Available
        for (int i = 0; i < rooms.length; i++) {
            for (int j = 0; j < rooms[i].length; j++) {
                rooms[i][j] = 'A';
            }
        }

        int choice;

        do {
            System.out.println("\n===== HOTEL ROOM BOOKING SYSTEM =====");
            System.out.println("1. View Room Status");
            System.out.println("2. Book a Room");
            System.out.println("3. Exit");
            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    // Display room status
                    System.out.println("\nRoom Status:");
                    System.out.println("A = Available, B = Booked");

                    for (int i = 0; i < rooms.length; i++) {
                        System.out.print("Floor " + (i + 1) + ": ");

                        for (int j = 0; j < rooms[i].length; j++) {
                            System.out.print("Room " + (j + 1) + "[" + rooms[i][j] + "] ");
                        }

                        System.out.println();
                    }
                    break;

                case 2:
                    // Book a room
                    System.out.print("Enter floor number (1-3): ");
                    int floor = sc.nextInt();

                    System.out.print("Enter room number (1-5): ");
                    int room = sc.nextInt();

                    // Validate floor and room number
                    if (floor < 1 || floor > 3 || room < 1 || room > 5) {
                        System.out.println("Invalid floor or room number.");

                    } else {
                        // Convert user input to array index
                        int floorIndex = floor - 1;
                        int roomIndex = room - 1;

                        if (rooms[floorIndex][roomIndex] == 'A') {
                            rooms[floorIndex][roomIndex] = 'B';
```

```
                            System.out.println("Room " + room + " on Floor " + floor
                                + " has been booked successfully.");

                        } else {
                            System.out.println("Sorry! The selected room is already booked.");
                        }
                    }
                    break;

                case 3:
                    System.out.println("Thank you for using the Hotel Room Booking System.");
                    break;

                default:
                    System.out.println("Invalid choice. Please select 1, 2 or 3.");
            }

        } while (choice != 3);

        sc.close();
    }
}
```

Output

